HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Connect Four Online

Sub: Network Programming

Members:

1. Nguyen Duc Hoang - 20235938
2. Tran Trung Hieu - 20235934
3. Pham Song Hao - 20235930

ONE LOVE. ONE FUTURE.

1.Project Idea and Objectives

Game concept:

Connect Four game: Two players take turns dropping discs into a board column, and the first to get four in a row (horizontal, vertical, or diagonal) wins. A full board without a winner is a draw.

Players register and log in, see who is online and their status (IDLE, IN_ROOM, IN_GAME), then either create or join rooms or challenge an idle player. Games can end by a win, a draw, a resignation, a timeout, or a disconnection.

1.Project Idea and Objectives

Core technical objective:

Protocol: Clients (Python) and the server (C) exchange commands, responses, and events as newline-delimited JSON (NDJSON) over TCP, with stream buffering so partial and merged messages are handled correctly.

Concurrency: The server handles many simultaneous clients in a single process using non-blocking sockets and poll() I/O multiplexing.

Real-time updates: The server pushes events (player status changes, challenges, moves, game results) to clients without polling.

Connection health: Periodic PING keep-alive treats connections silent for 45 seconds as disconnected, and a mid-game disconnect counts as a forfeit.

2. Phase 1 Scope

1. Define client and server responsibilities
2. Select TCP and the NDJSON message format
3. Design module boundaries and the folder structure
4. Define communication interfaces between modules
5. Plan testing with a CLI client and a mock server

3. Intended Features

1. **Account Management**: User registration, login/logout, password change, and basic profiles displaying ELO rating, wins, losses, and draws. Passwords will be salted and hashed.
2. **Online Players List**: A real-time updated roster showing active players, their current ELO, and live status.
3. **Game Lobby**: Room management functionality allowing players to create, join, leave, and view open rooms along with current player counts.
4. **Challenge System**: Direct invites between players; idle players can send, accept, or decline challenge requests, with pending invitations expiring after 30 seconds.

3. Intended Features

5. **Authoritative Move Validation**: Server-side game validation to ensure moves are made only during a player's turn, within board column bounds, and inside columns that are not already full.
6. **Win/Draw Detection**: Automatic server-side evaluation detecting 4-in-a-row connections (horizontal, vertical, diagonal) or full-board stalemates, followed by match-result notifications.
7. **Forfeit / Resignation**: Allows players to manually resign mid-match, and automatically treats disconnections as forfeits.
8. **Graphical Interface (GUI)**: An interactive visual interface featuring column-click moves on the board, lobby navigation, active player views, match controls, and replay screens.

5. Client–Server Architecture

I. Python Client

1. Display authentication screens, the lobby, and the game board
2. Capture player actions
3. Send requests and update the interface using server data

II. C Server

1. Manage connections, accounts, and game rooms
2. Maintain the board and current turn
3. Validate moves and determine game results
4. Access PostgreSQL to store persistent data

=> **Core principle**: The server maintains the authoritative game state.

Client–Server model:

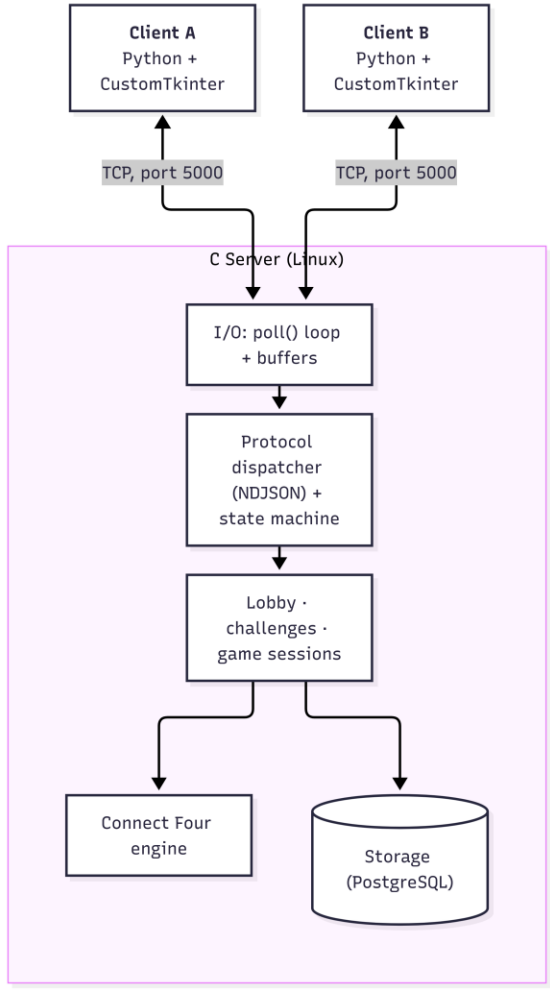
- Clients never talk to each other; every message goes through the server.
- One persistent TCP connection per client (port 5000).

Server (C, Linux):

- I/O layer: poll() loop, non-blocking sockets, per-client input/output buffers.
- Protocol layer: NDJSON line parser (cJSON), message dispatcher, connection state machine.
- Game layer: lobby, rooms, challenges, game sessions, turn timers.
- Connect Four engine (6x7 board, move validation, win/draw detection) and storage (accounts, ELO, history, move logs).

Client (Python):

Network thread receives and splits NDJSON lines, passes them to the CustomTkinter GUI through an event queue.



7. TCP Stream Handling and Move Synchronization

I. TCP stream handling

1.One recv() may return part of a message or several messages
2.Server and client each keep a per-connection receive buffer
3.Only complete \n-terminated lines are parsed; the rest stays in the buffer
4.Oversized or malformed lines return ERR without dropping the connection

II. Move processing and synchronization

1.Client sends only the chosen column
2.Server checks turn, column range and free space
3.Server updates the board, then checks win or draw
4.Server replies OK to the mover and sends OPP_MOVE to the opponent
5.Invalid move: ERR, game state unchanged. The client redraws only after the server confirms

8. Application Protocol: NDJSON

Item	Rule
Encoding	UTF-8. Protocol values (username, command names, status codes) use ASCII only
Message boundary	Each message is one line terminated by \n (\r\n is accepted; \r is stripped)
Line content	Exactly one JSON object (not an array, string or standalone number)
Newlines	JSON must be compact, with no literal newline characters inside. Newlines inside strings must be escaped as \n (the JSON library does this automatically)

8. Application Protocol: NDJSON

Maximum length	4096 bytes per line, including the \n. Longer line: the server returns ERR 108 and discards the entire line (reads and drops everything up to the next \n)
Required fields	Every message has a "type" field (string). Every client command additionally has an "id" (positive integer)
Command / event names	UPPERCASE, e.g. "LOGIN"
Unknown fields	The receiver ignores fields it does not recognize (so new fields can be added without breaking older versions)
Username	3 to 16 characters, only A-Z a-z 0-9 _
Password	6 to 32 characters, no spaces

8. Application Protocol: NDJSON

Move request:

```
{"type":"MOVE","id":5,"col":3}
```

Response to the player:

```
{"type":"OK","id":5,"cmd":"MOVE","col":3,"row":0}
```

Event sent to the opponent:

```
{"type":"OPP_MOVE","col":3,"row":0}
```

9. Proposed Technology Stack

Component	Technology	Purpose
Server	C, non-blocking sockets, poll()	Manage connections and process client requests
Client	Python	Implement the interface and player interactions
Proposed GUI	CustomTkinter	Build authentication, lobby, and board screens
Communication	TCP + NDJSON	Exchange commands, responses, and events
C JSON library	cJSON	Parse and validate messages
Database	PostgreSQL	Store accounts, matches, and move logs
Server build	Makefile	Compile the C modules

10. Team Responsibilities and Integration Plan

Responsibility	Scope
Tran Trung Hieu	Networking, poll(), sessions, lobby, and authentication handlers
Nguyen Duc Hoang	Database, game engine, match lifecycle, and ELO
Pham Song Hao	Client GUI, network client, mock server, and replay
Entire team	Protocol specification, integration, documentation, and demo

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !